

Task 1:

1. Linear Regression:

linear regression is expansion of straight line equation i.e $y = mx + c$. When it comes to machine learning model, straight line equation is transformed as weight sum of input x and intercept.

And straight line is called best fit line in linear regression.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- $\hat{y}$ is the predicted value.
 - n is the number of features.
 - x_i is the i^{th} feature value.
 - θ_j is the j^{th} model parameter (including the bias term θ_0 and the feature weights $\theta_1, \theta_2, \dots, \theta_n$).
-

And $\hat{y}$ is predicted value or we can say hypothesis.

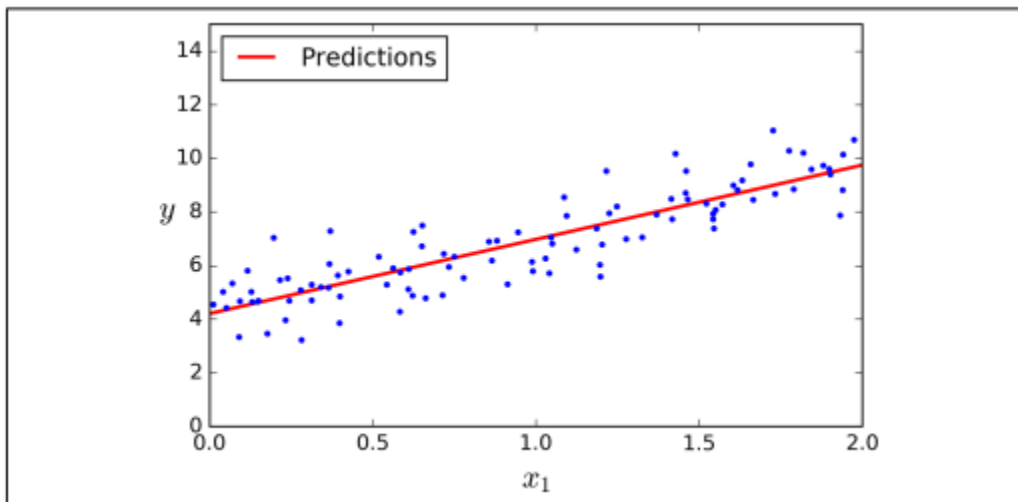
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Here also,

θ_0 is the point in y that best fit line intercept as $x = 0$.

θ_1 is slope that tells for each unit increase in x the predicted value of y increase/decrease by θ_1 unit.

Geometrical intuition and creation of best fit line:



Here blue dots denotes actual values and best fit line denotes predicted value. Best fit line is what above hypothesis function creates.

For better result the best fit line should cover greater number of actual values. That can be measured by **residual error**. Residual error is the difference of actual values and predicted value at given instance. The model makes best fit line where sum of squared residuals is minimum.

Residual:

$$e_i = y_i - \hat{y}_i$$

Sum of squared residuals:

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

We, should minimize the average sum of squared residuals to get best result. Which is called **cost function**.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

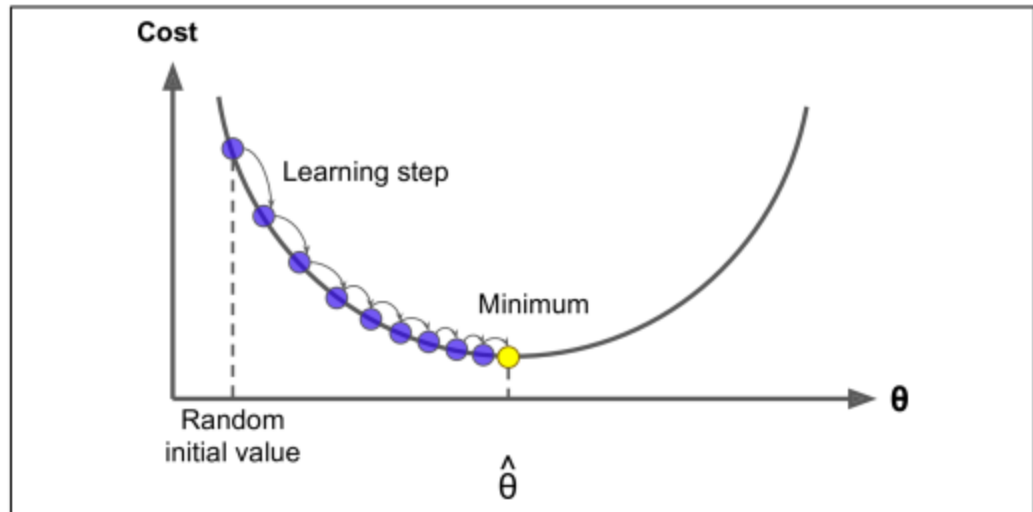
- $J(\theta)$ is the **cost function** (or **loss function**).
- The goal of linear regression is to find $\theta_0, \theta_1, \dots$ that **minimize** this cost.

Main goal of squared residuals is to avoid negative values, and square penalizes more to high strength error.

When we plot J , against θ_0 and θ_1 it gives bowl shaped surface. And to minimize cost function it uses Gradient Descent inside of Repeat convergence algorithm.

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

Where, α is learning rate: step taken towards minimum.



This curve is gradient descent plotted on a cost surface, we start at a random point on the cost surface then gradient descent moves step by step at learning rate towards the minimum.

Learning rate should be not much high, that will make gradient descent to overshoot minimum point and miss minimum value. Which will make model to not converge to minimum and model fails to predict.

So, starting learning rate around 0.001 to 0.1 is common in practice.

The gradient descent get computed and theta is adjusted until it converges to minimum, which will give minimum cost function. Ultimately linear regression generates best fit line.

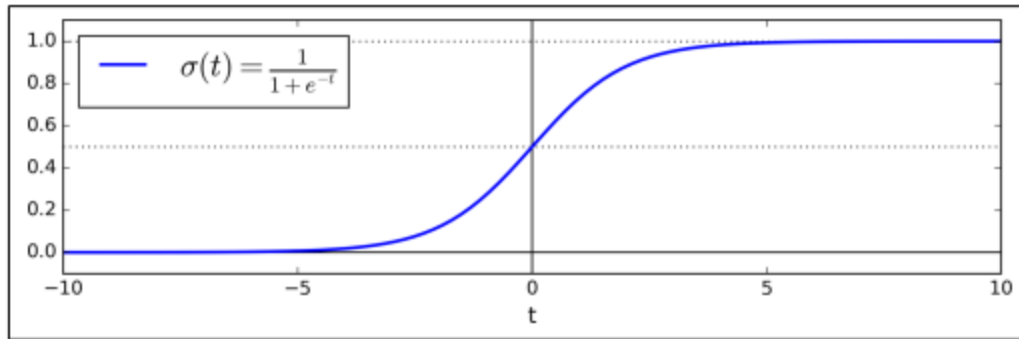
Logistic regression:

- It is used for classification problem, an extension of linear regression.
- linear regression predict continuous value, but it predicts probability of binary outcome. i.e 0 or 1.
- It starts with weighted sum of inputs.

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

- To predict probability it uses sigmoid function:

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



When we plot this hypothesis, we will get S-shaped curve from 0 to 1 limits $z \rightarrow +\infty$ and 0 as $z \rightarrow -\infty$.

Once the Logistic Regression model has estimated the probability $p = h_{\theta}(x)$ that an instance x belongs to the positive class, it can make its prediction $\hat{y}$ easily.

- In classification residuals and cost function is not squared. we use log loss to measure prediction error.

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

-
- Similar to linear regression we use gradient descent to minimize cost function. But error is not squared.

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Learning controls the step size, it should not be too high and too low so that it iteratively updates theta until cost converges to minimum. Which gives best fit s curve for classification.